DIGA — Server ML/DL per la Didattica

1. Hardware del server

Componente	Modello	Note
Case	Corsair 4000D RGB	—
Alimentatore	ASUS TUF Gaming 1000W	Sufficiente per RTX 5090 (TDP ~575W)
Scheda madre	MSI MAG X870 Tomahawk WiFi	Socket AM5, PCIe 5.0
CPU	AMD Ryzen 9 9950X	16 core / 32 thread — gestione container + pre-processing
Dissipatore	Arctic Liquid Freezer III Pro 360	AIO 360mm
RAM	128 GB DDR5 6000 MHz	4×32 GB Corsair Vengeance
SSD	Samsung 990 PRO 1 TB (MZ-V9P1T0BW)	NVMe PCIe 4.0 — OS + dati
GPU	Gigabyte RTX 5090 32 GB	Blackwell, CUDA — dedicata esclusivamente al training

Requisiti software lato host

- **Sistema operativo**: Proxmox VE (basato su Debian)
- **NVIDIA Driver** compatibile con RTX 5090 (serie 570+)
- **NVIDIA Container Toolkit** per il passthrough GPU ai container

---

2. Architettura generale

Il server ospita due tipi di container gestiti da Proxmox:

1. **Container studente (LXC, solo CPU)** — uno per ogni gruppo/progetto, con JupyterLab per scrivere codice, esplorare dati e fare pre-processing.
2. **Container/meccanismo GPU** — accesso esclusivo alla RTX 5090 per eseguire i training. Lo studente non accede mai direttamente alla GPU.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      PROXMOX VE (host)                               │
│                                                                      │
│  /srv/datasets/        (dataset condivisi, read-only)                │
│  /srv/students/        (cartelle di lavoro studenti)                 │
│  /srv/jobs/            (coda job training — file JSON)               │
│  /srv/scripts/         (script di training, read-only)               │
│  /srv/guidelines/      (guide e regolamenti, read-only)              │
│  /srv/logs/            (log di training)                             │
│                                                                      │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐                 │
│  │ Container    │ │ Container    │ │ Container    │  ...            │
│  │ Studente A   │ │ Studente B   │ │ Studente C   │                 │
│  │ (CPU only)   │ │ (CPU only)   │ │ (CPU only)   │                 │
│  │ JupyterLab   │ │ JupyterLab   │ │ JupyterLab   │                 │
│  └──────────────┘ └──────────────┘ └──────────────┘                 │
│                                                                      │
│  ┌──────────────────────────────────────────────┐                   │
│  │  GPU Worker (RTX 5090)                       │                   │
│  │  Esegue i job dalla coda /srv/jobs/          │                   │
│  └──────────────────────────────────────────────┘                   │
└──────────────────────────────────────────────────────────────────────┘
```

---

3. Struttura del filesystem sul server

3.1 Cartelle principali

```
/srv/
├── datasets/                          # Dataset condivisi (read-only nei
container)
│   ├── cats_dogs/
│   ├── recycling/
│   └── film_reviews/
├── students/                          # Una sottocartella per
gruppo/progetto
│   ├── 5IA_cnn_rossi/
│   ├── 5IA_nlp_bianchi/
│   └── 4IA_classify_verdi/
├── jobs/                              # Coda dei job di training (file
.json)
├── scripts/                           # Script di training (read-only)
```

```
├── guidelines/                        # Guide per gli studenti (read-only)
└── logs/                              # Log dei training eseguiti
```

3.2 Convenzione di denominazione progetti

```
<classe>_<progetto>_<cognome_responsabile>
```

Esempi: `5IA_cnn_rossi`, `4IA_sentiment_bianchi`

> **Da valutare**: una convenzione migliore

3.3 Mount points nel container studente

Percorso host	Percorso container	Permessi
`/srv/students/<progetto>`	`/home/studente/work`	lettura/scrittura
`/srv/datasets`	`/datasets`	solo lettura
`/srv/guidelines`	`/guidelines`	solo lettura
`/srv/scripts`	`/scripts`	solo lettura

Dentro JupyterLab lo studente vede:

```
work/          ← i propri notebook e file
datasets/      ← dataset condivisi (read-only)
guidelines/    ← documentazione e guide
scripts/       ← script di training (read-only)
```

---

4. Container studente (LXC, solo CPU)

Ogni gruppo di studenti riceve un container LXC unprivileged con le seguenti caratteristiche.

4.1 Lo studente può

- Leggere i dataset condivisi
- Scrivere notebook e file nella propria cartella di lavoro
- Fare pre-processing dei dati (generare file `.npy` dai dati grezzi)
- Sottomettere job di training creando un file JSON nella coda

4.2 Lo studente NON può

- Accedere alla GPU
- Cancellare o creare dataset
- Aprire un terminale shell da JupyterLab
- Avviare direttamente il training sulla GPU

4.3 Limiti di risorse per container

Risorsa	Limite suggerito	Note
CPU cores	2	Sufficienti per pre-processing
RAM	4 GB	Adeguata per manipolazione dati
Disco	10 GB	Notebook + file intermedi
GPU	**nessuna**	Mai esposta al container studente
Terminale shell	**disabilitato**	`NotebookApp.terminals_enabled = False`
PID limit	200	Evita fork bomb

4.4 Creazione container LXC con Proxmox

```
# Crea il container LXC unprivileged
pct create <VMID> local:vztmpl/ubuntu-22.04-standard_22.04-1_amd64.tar.zst \
  --hostname jupyter-<progetto> \
  --cores 2 \
  --memory 4096 \
  --rootfs local-lvm:10 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp \
  --mp0 /srv/students/<progetto>,mp=/home/studente/work \
  --mp1 /srv/datasets,mp=/datasets,ro=1 \
  --mp2 /srv/guidelines,mp=/guidelines,ro=1 \
  --mp3 /srv/scripts,mp=/scripts,ro=1 \
  --unprivileged 1

# Avvia il container
pct start <VMID>

# Installa JupyterLab + dipendenze (senza GPU)
pct exec <VMID> -- bash -c "
  apt-get update && apt-get install -y python3-pip python3-venv &&
  pip3 install jupyterlab numpy pandas matplotlib scikit-learn &&
  pip3 install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
"
```

> **Nota**: si installa PyTorch **CPU-only** nel container studente. Serve per definire il modello e fare pre-processing, ma non può eseguire training su GPU.
>
> > **Da valutare**: permettere di usare più librerie di Deep Learning o solo una? Meglio Keras?

4.5 Servizio systemd per JupyterLab

```
pct exec <VMID> -- bash -c "cat > /etc/systemd/system/jupyter.service <<
'EOF'
[Unit]
Description=JupyterLab
After=network.target

[Service]
User=root
ExecStart=/usr/local/bin/jupyter lab \
  --ip=0.0.0.0 \
  --port=8888 \
  --no-browser \
  --allow-root \
  --NotebookApp.terminals_enabled=False
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload &&
systemctl enable jupyter &&
systemctl start jupyter"
```

JupyterLab è raggiungibile su `http://<IP-container>:8888`. Per trovare l'IP: `pct exec <VMID> -- ip a`.

> **Da valutare**: usare **JupyterHub** (singola istanza, autenticazione multi-utente) al posto di tanti JupyterLab separati, per semplificare la gestione dei container.

---

5. Gestione del training GPU — Due soluzioni a confronto

Il problema centrale: gli studenti devono poter lanciare training sulla GPU senza accedervi direttamente, e la GPU (una sola RTX 5090) deve essere condivisa in modo ordinato tra tutti i

progetti.

Si presentano **due soluzioni alternative**. Entrambe partono dallo stesso presupposto: lo studente prepara un file `config.json` con i parametri del training.

5.0 Il file config.json (comune a entrambe le soluzioni)

Lo studente crea un file JSON che descrive il training da eseguire:

```
{
  "progetto": "5IA_cnn_rossi",
  "modello": "work/model.py",
  "dataset": "cats_dogs",
  "dati_preprocessati": "work/data.npy",
  "labels_preprocessati": "work/labels.npy",
  "epochs": 15,
  "batch_size": 32,
  "learning_rate": 0.001,
  "output_dir": "work/results/"
}
```

Vincoli di validazione (applicati in entrambe le soluzioni)

Parametro	Limite	Motivazione
`epochs`	max 20	Evitare training infiniti
`batch_size`	max 64	Limitare consumo VRAM - questa causa CUDA OUT OF MEMORY
`learning_rate`	range [1e-5, 0.1]	Evitare divergenza
Tempo massimo	30 min per job	Garantire turnover sulla GPU

5.1 Soluzione A — Script di training + coda shell per GPU

Principio: lo studente lancia dal proprio notebook uno script Python (`run_training.py`) fornito in read-only. Questo script valida i parametri, si mette in coda per la GPU con uno script shell (`wait_gpu.sh`), e poi esegue il training.

Flusso

```
┌──────────────────┐      ┌──────────────────────┐      ┌──────────────┐
│  Notebook stud.  │      │   run_training.py     │      │  GPU (host)  │
│                  │──────►│  (in /scripts, r/o)   │──────►│              │
│ !python          │      │                       │      │  Training    │
│ /scripts/run_    │      │ 1. Valida config.json │      │  effettivo   │
│ training.py      │      │ 2. wait_gpu.sh (coda) │      │              │
│ config.json      │      │ 3. Limita VRAM        │      │              │
│                  │      │ 4. Lancia training    │      │              │
│                  │      │ 5. Scrive log         │      │              │
└──────────────────┘      └──────────────────────┘      └──────────────┘
```

Cosa fa `run_training.py`

1. **Carica e valida** `config.json` — controlla epoche, batch size, path
2. **Chiama** `wait_gpu.sh` — script shell che verifica se la GPU è occupata (tramite `nvidia-smi`) e attende il proprio turno con un meccanismo di lock file
3. **Limita la memoria GPU** via PyTorch o librerie deep learning **(si può fare in Keras?)**:

   ```
   import torchtorch.cuda.set_per_process_memory_fraction(0.5)  # max 16 GB su 32 GB
   ```

4. **Esegue il training** importando il modello dello studente
5. **Salva il modello** nella directory di output specificata
6. **Scrive un log** in `/srv/logs/` con timestamp, progetto, parametri, esito

Esempio di `wait_gpu.sh`

```bash
#!/bin/bash
LOCKFILE="/tmp/gpu.lock"

while ! mkdir "$LOCKFILE" 2>/dev/null; do
    echo "GPU occupata, attendo..."
    sleep 10
done

# Il lock è acquisito — il training può partire
# Il lock viene rilasciato da run_training.py alla fine
trap "rmdir $LOCKFILE" EXIT
```

Pro e Contro

Pro	Semplice da implementare; lo studente ha feedback immediato nel notebook

	(vede l'output del training in tempo reale); non serve un container GPU separato
Contro	Lo script gira nel contesto dell'host o di un container con accesso GPU — richiede attenzione ai permessi; il notebook dello studente resta "bloccato" durante il training; la coda basata su lock file è rudimentale (no priorità, no fairness)

Requisito

Il container studente **oppure** l'host deve poter accedere alla GPU per questa soluzione. In alternativa, `run_training.py` può essere eseguito sull'host tramite un meccanismo di invocazione remota (es. il container scrive il config.json in `/srv/jobs/` e un **cron job** sull'host lo raccoglie — in questo caso la soluzione converge verso la Soluzione B).

5.2 Soluzione B — Container GPU Worker con coda job

Principio: un container (o processo host) dedicato con accesso esclusivo alla GPU esegue un loop continuo che consuma job dalla coda. Lo studente non lancia nulla direttamente: copia il file `config.json` nella cartella `/srv/jobs/` e attende il risultato.

Flusso

```
┌──────────────────┐      ┌──────────────┐      ┌──────────────────────┐
│  Notebook stud.  │      │  /srv/jobs/  │      │  GPU Worker Container  │
│                  │──────►│              │──────►│                      │
│  Copia config    │      │  config_5IA_ │      │  Loop continuo:      │
│  .json nella     │      │  cnn_rossi_  │      │  1. Cerca job in /jobs │
│  cartella jobs   │      │  20260316_   │      │  2. Valida parametri  │
│                  │      │  1430.json   │      │  3. Esegue training   │
│                  │      │              │      │  4. Salva risultati   │
│  Attende         │      │              │      │  5. Rimuove job       │
│  risultato in    │      │              │      │  6. Scrive log        │
│  work/results/   │      │              │      │                      │
└──────────────────┘      └──────────────┘      └──────────────────────┘
```

Sottomissione del job dallo studente

Lo studente esegue nel proprio notebook:

```
import json, shutil, datetime

config = {
    "progetto": "5IA_cnn_rossi",
```

```
    "modello": "work/model.py",
    "dataset": "cats_dogs",
    "dati_preprocessati": "work/data.npy",
    "labels_preprocessati": "work/labels.npy",
    "epochs": 15,
    "batch_size": 32,
    "learning_rate": 0.001,
    "output_dir": "work/results/"
}

# Nome file univoco con timestamp
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
nome_file = f"job_{config['progetto']}_{timestamp}.json"

with open(f"/srv/jobs/{nome_file}", "w") as f:
    json.dump(config, f, indent=2)

print(f"Job sottomesso: {nome_file}")
print("Controlla work/results/ per i risultati.")
```

> **Nota**: la cartella `/srv/jobs/` deve essere montata in lettura/scrittura nel container studente (mount point aggiuntivo).

Script del GPU Worker (`gpu_worker.py`)

```
import os
import time
import json
import logging
import torch
from datetime import datetime

# Configurazione
JOBS_DIR = "/srv/jobs"
LOGS_DIR = "/srv/logs"
MAX_EPOCHS = 20
MAX_BATCH_SIZE = 64
MAX_TIME_SECONDS = 1800  # 30 minuti
GPU_MEMORY_FRACTION = 0.8

logging.basicConfig(
    filename=os.path.join(LOGS_DIR, "gpu_worker.log"),
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
```

```python
def validate_config(config):
    """Valida i parametri del job prima di eseguirlo."""
    if config.get("epochs", 0) > MAX_EPOCHS:
        raise ValueError(f"Troppe epoche: {config['epochs']} (max
{MAX_EPOCHS})")
    if config.get("batch_size", 0) > MAX_BATCH_SIZE:
        raise ValueError(f"Batch troppo grande: {config['batch_size']} (max
{MAX_BATCH_SIZE})")
    # Verificare che i file esistano
    for key in ["dati_preprocessati", "labels_preprocessati", "modello"]:
        path = os.path.join(f"/srv/students/{config['progetto']}",
config[key])
        if not os.path.exists(path):
            raise FileNotFoundError(f"File non trovato: {path}")

def run_training(config):
    """Esegue il training con i parametri specificati."""
    # Limita memoria GPU
    torch.cuda.set_per_process_memory_fraction(GPU_MEMORY_FRACTION)

    # ... importa modello, carica dati, esegui training ...
    # (implementazione specifica per il framework scelto)

    logging.info(f"Training completato per {config['progetto']}")

# === LOOP PRINCIPALE ===
logging.info("GPU Worker avviato")

while True:
    jobs = sorted(os.listdir(JOBS_DIR))  # ordine alfabetico = FIFO per
timestamp

    if len(jobs) > 0:
        job_file = jobs[0]
        job_path = os.path.join(JOBS_DIR, job_file)

        logging.info(f"Inizio job: {job_file}")

        try:
            with open(job_path) as f:
                config = json.load(f)

            validate_config(config)
            run_training(config)
```

```
            logging.info(f"Job completato: {job_file}")

        except Exception as e:
            logging.error(f"Job fallito ({job_file}): {e}")
            # Salvare errore nella cartella dello studente
            error_path = os.path.join(
                f"/srv/students/{config.get('progetto', 'unknown')}",
                config.get("output_dir", ""),
                "error.log"
            )
            with open(error_path, "w") as ef:
                ef.write(f"Errore nel training: {e}\n")

        finally:
            os.remove(job_path)

    time.sleep(5)
```

Container GPU Worker in Proxmox

```
# Creare un container privilegiato con accesso GPU
# Opzione 1: LXC con GPU passthrough
pct create <VMID_GPU> local:vztmpl/ubuntu-22.04-standard_22.04-
1_amd64.tar.zst \
  --hostname gpu-worker \
  --cores 4 \
  --memory 32768 \
  --rootfs local-lvm:20 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp \
  --mp0 /srv/students,mp=/srv/students \
  --mp1 /srv/datasets,mp=/srv/datasets,ro=1 \
  --mp2 /srv/jobs,mp=/srv/jobs \
  --mp3 /srv/logs,mp=/srv/logs \
  --mp4 /srv/scripts,mp=/srv/scripts,ro=1

# Configurare GPU passthrough (in /etc/pve/lxc/<VMID_GPU>.conf):
# lxc.cgroup2.devices.allow: c 195:* rwm
# lxc.cgroup2.devices.allow: c 509:* rwm
# lxc.mount.entry: /dev/nvidia0 dev/nvidia0 none bind,optional,create=file
# lxc.mount.entry: /dev/nvidiactl dev/nvidiactl none
bind,optional,create=file
# lxc.mount.entry: /dev/nvidia-uvm dev/nvidia-uvm none
bind,optional,create=file
```

> **Alternativa Docker**: se si preferisce Docker al posto di LXC per il worker GPU:

```
docker run -d \
  --name gpu-worker \
  --gpus all \
  --cpus=4 \
  --memory=32g \
  -v /srv/students:/srv/students \
  -v /srv/datasets:/srv/datasets:ro \
  -v /srv/jobs:/srv/jobs \
  -v /srv/logs:/srv/logs \
  -v /srv/scripts:/srv/scripts:ro \
  gpu-worker-image python gpu_worker.py
```

Servizio systemd per il GPU Worker

```
[Unit]
Description=GPU Worker - DIGA Training Queue
After=network.target

[Service]
ExecStart=/usr/bin/python3 /srv/scripts/gpu_worker.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Pro e Contro

Pro	Isolamento completo: lo studente non tocca mai la GPU; coda ordinata (FIFO con timestamp); il notebook dello studente resta libero dopo la sottomissione; più facile da monitorare e loggare; non serve esporre la GPU a nessun container studente
Contro	Più complesso da configurare; lo studente non vede l'output in tempo reale (deve controllare periodicamente i risultati); serve un meccanismo di notifica (opzionale) per avvisare che il training è completato

---

5.3 Confronto sinottico

Aspetto	Soluzione A (Script + Shell)	Soluzione B (GPU Worker)
Complessità setup	Bassa	Media
Isolamento GPU	Parziale (serve accesso)	Completo
Feedback studente	Tempo reale nel notebook	Asincrono (controlla risultati)
Coda	Lock file (rudimentale)	FIFO su filesystem (robusta)
Notebook durante training	Bloccato	Libero
Logging	Nello script	Centralizzato nel worker
Scalabilità	Limitata	Buona (estendibile a più GPU)
Rischio sicurezza	Medio (accesso GPU esposto)	Basso

Raccomandazione: la **Soluzione B** è preferibile in un contesto scolastico multi-utente perché garantisce isolamento completo, coda ordinata e logging centralizzato. La Soluzione A può essere utile in fase di prototipazione iniziale per la sua semplicità.

---

6. Limitazione dell'uso GPU

6.1 Via codice (PyTorch)

```
import torch
# Limita al 50% della VRAM (16 GB su 32 GB)
torch.cuda.set_per_process_memory_fraction(0.5)
```

6.2 Via validazione del notebook (opzionale)

Script che analizza il notebook dello studente prima di accettare il job, per verificare che non contenga chiamate dirette alla GPU o import non autorizzati:

```
python /srv/scripts/validate_notebook.py <notebook.ipynb>
```

6.3 Timeout per job

Entrambe le soluzioni devono implementare un timeout massimo per job (es. 30 minuti) per evitare che un training difettoso monopolizzi la GPU indefinitamente.

7. Monitoraggio

7.1 Stato della GPU

```
# Dall'host Proxmox
nvidia-smi

# Monitoraggio continuo (aggiorna ogni 2 secondi)
watch -n 2 nvidia-smi
```

7.2 Logging

Tutti i training vengono registrati in `/srv/logs/` con:

- Timestamp di inizio e fine
- Progetto e studente
- Parametri del training
- Esito (successo/errore)
- Metriche finali (loss, accuracy)

7.3 TensorBoard (opzionale)

Possibilità di attivare un container separato con TensorBoard per visualizzare le curve di training di tutti i progetti:

```
tensorboard --logdir=/srv/logs/tensorboard --host=0.0.0.0 --port=6006
```

Accessibile sulla rete locale della scuola via `http://<IP-server>:6006`.

---

8. Workflow completo dello studente

```
1. APPROVAZIONE     Il docente approva il progetto e crea il container
                     ↓
2. ESPLORAZIONE     Lo studente accede a JupyterLab, esplora i dataset
                     in /datasets (read-only)
                     ↓
3. PRE-PROCESSING   Scrive un notebook per preparare i dati:
                     carica immagini/testi → genera file .npy
                     Salva in work/
```

```
                        ↓
4. MODELLO          Definisce l'architettura della rete neurale
                     in un file Python (es. work/model.py)
                        ↓
5. CONFIG           Crea config.json con i parametri del training
                        ↓
6. SOTTOMISSIONE    • Soluzione A: !python /scripts/run_training.py
config.json
                     • Soluzione B: copia config.json in /srv/jobs/
                        ↓
7. TRAINING         La GPU esegue il training (lo studente attende)
                        ↓
8. RISULTATI        Modello addestrato e log salvati in work/results/
                     Lo studente analizza i risultati nel notebook
```

---

9. Approvazione e creazione progetti

9.1 Procedura

All'approvazione di un nuovo progetto si devono eseguire i seguenti passi:

1. Creare la cartella `/srv/students/<classe>_<progetto>_<cognome>`
2. Creare il container LXC con i mount point corretti
3. Installare JupyterLab e le dipendenze
4. Configurare e avviare il servizio systemd
5. Comunicare allo studente IP e token di accesso

9.2 App web di approvazione (da sviluppare)

Funzionalità previste:

- Form per richiesta progetto (nome, classe, descrizione, framework scelto)
- Notifica via email ai docenti per approvazione
- Alla conferma: creazione automatica di cartelle e container (via script)
- Dashboard con stato dei progetti attivi

---

10. Questioni aperte (TODO)

#	Questione	Note
1	**JupyterLab vs JupyterHub**	JupyterHub semplificherebbe la gestione multi-utente con un singolo punto di accesso e autenticazione centralizzata, eliminando la necessità di un container per studente
2	**Upload dataset da parte degli studenti**	Permetterlo? Se sì, serve una cartella intermedia con approvazione docente prima di spostare in `/srv/datasets/`
3	**Framework vincolato?**	Imporre solo PyTorch? Oppure supportare anche Keras/TensorFlow? (impatta le dipendenze nel worker GPU)
4	**Notifica completamento training**	Meccanismo per avvisare lo studente (email, file flag, webhook)
5	**Backup**	Strategia di backup per notebook studenti e modelli addestrati (SSD singolo = rischio)
6	**Test pre-requisito per training**	Richiedere agli studenti di superare un quiz prima di poter sottomettere job GPU?
7	**Gestione container con Proxmox**	Automatizzare creazione/rimozione container tramite le API di Proxmox
8	**Storage aggiuntivo**	1 TB potrebbe non bastare per molti dataset + modelli. Prevedere espansione.